TECH REFERENCE

Causal Expert & Optimal Transport Expert

Technical Reference: NOTEARS Causal Inference and Sinkhorn Optimal Transport

Author 1[1], Author 2[1]

[1]*Organization Name*
contact@org.com

Scope.

- Causal Expert: learns a causal DAG among features using Pearl SCM + NOTEARS acyclicity constraint
- OT Expert: Sinkhorn entropy-regularized optimal transport → customer↔prototype Wasserstein distances
- Shared input: same normalized features, different mathematical extraction (causal direction vs. distributional geometry)
- Numerical-stability issues and mitigations under FP16 mixed-precision training

Design vs. Implementation Note. This document is written against the full-bank design (734D). The current Santander benchmark implementation uses 1211D input (17 feature groups, 2026-04-28). Causal expert: 129D; OT expert: 95D. Refer to `outputs/phase0/feature_schema.json` for the current routing indices.

Contents

Causal Expert

Pearl Causal Inference Background

The Three Stages of Causal Inference

Judea Pearl categorized the epistemological levels of causal inference into three stages.

Stage 1: Association. Identifies correlations among variables from observational data. $P(Y \mid X)$ — "How does the probability of Y change when X is observed?" Traditional recommendation systems operate at this level. The observation that "premium card holders have higher travel insurance enrollment rates" is an association; it does not identify causal direction.

Stage 2: Intervention. Estimates the effect of artificially manipulating a variable. $P(Y \mid \text{do}(X = x))$ — "If X is set to x, how does Y change?" At this level, the influence of confounding variables can be controlled. Example: holding income level as a confounder constant, estimate the pure causal effect of offering a premium card on insurance enrollment.

Stage 3: Counterfactual. "Would the outcome have been different if a different action had been taken?" $P(Y_x \mid X = x', Y = y')$ — estimates the individual treatment effect (ITE). In the recommendation context, this corresponds to "What would have happened if product B, rather than A, had been recommended to this customer?"

The Confounding Trap in Correlation

Relying on correlations in recommendation systems introduces vulnerability to confounding variables. In the structure "premium card ownership ← high income → travel insurance enrollment," high income is a confounder that influences both variables. A correlation-based system recommends travel insurance to premium card holders, but providing the card for free may not change the insurance enrollment rate at all.

The Causal Expert learns causal directions and strengths among variables via the adjacency matrix $\boldsymbol{W}$, embedding an intervention-level (Stage 2) reasoning structure into recommendations. $W^2_{i,j}$ encodes "how much does variable i change when variable j is intervened upon."

NOTEARS Continuous DAG Constraint

The Challenge of DAG Structure Learning

Learning directed acyclic graph (DAG) structure has traditionally been an NP-hard problem. The number of possible DAGs over d variables grows super-exponentially (approximately 4.2×10^{18} for $d = 10$). Existing constraint-based approaches (PC algorithm) and score-based approaches (GES) could not escape the combinatorial search bottleneck.

NOTEARS Core Idea

NOTEARS (Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning) by Zheng et al. (NeurIPS 2018) completely bypasses combinatorial search by transforming the acyclicity condition into a differentiable continuous equality constraint.

$$h(\boldsymbol{W}) = \operatorname{tr}\left(e^{\boldsymbol{W}\circ\boldsymbol{W}}\right) - d = 0 \tag{1}$$

Where:

- $e^{\boldsymbol{M}}$: matrix exponential
- $\operatorname{tr}(\cdot)$: trace
- d: number of causal variables (in this implementation, $d = 32$)
- $\boldsymbol{W} \circ \boldsymbol{W}$: Hadamard (element-wise) square, guaranteeing non-negative causal strength

Mathematical interpretation. The (i, i) entry of the k-th power $\boldsymbol{A}^k$ is the weighted count of length-k paths from node i back to itself. Therefore, the diagonal entry $\left(e^{\boldsymbol{A}}\right)_{i,i}$ of the matrix exponential $e^{\boldsymbol{A}} = \sum_{k=0}^{\infty} \frac{\boldsymbol{A}^k}{k!}$ is the weighted sum of all cyclic paths of any length returning to node i.

In a DAG, no cyclic paths exist, so the diagonal entries of all terms with $k \geq 1$ are zero. Only the $k = 0$ (identity matrix) contribution of 1 remains, giving $e^{\boldsymbol{A}}_{i,i} = 1$ and $\operatorname{tr}\left(e^{\boldsymbol{A}}\right) = d$. Therefore $h(\boldsymbol{W}) = 0$ guarantees the graph is a DAG.

Taylor 10-Term Approximation

Direct computation of the matrix exponential requires $O(d^3)$ operations such as eigendecomposition. The implementation approximates by accumulating terms $k = 1$ through $k = 10$ of the Taylor series.

$$e^{\boldsymbol{M}} = \boldsymbol{I} + \sum_{k=1}^{\infty} \frac{\boldsymbol{M}^k}{k!} \approx \boldsymbol{I} + \sum_{k=1}^{10} \frac{\boldsymbol{M}^k}{k!} \tag{2}$$

In the actual implementation, the $k = 0$ term (identity matrix) is not included in the loop; the $-d$ subtraction in $h(\boldsymbol{W}) = \operatorname{tr}\left(e^{\boldsymbol{M}}\right) - d$ eliminates the identity matrix contribution. The loop accumulates $k = 1$ through $k = 10$:

```
M_power = torch.eye(d, device=W.device, dtype=W.dtype)
for i in range(1, 11):       # i = 1, 2, ..., 10  (k=1..10)
    M_power = M_power @ W_sq / i  # k! is constructed automatically
    h = h + torch.trace(M_power)  # accumulate tr(M^k / k!)
# h corresponds to tr(e^M) - d (equivalent to subtracting the trace of the k=0 identity term)
```

Ten terms suffice to detect all cyclic paths of length up to 10; in a 32-node DAG, cycles of more than 10 hops are practically unlikely. When the elements of $\boldsymbol{W}$ are small (initialized at 0.01), higher-order terms decay rapidly, providing sufficient practical precision.

SCM (Structural Causal Model) Intervention

Feature Compressor

A bottleneck module that projects high-dimensional features into the DAG variable space.

$$\boldsymbol{z} = \text{Compressor}(\boldsymbol{x}): \quad \mathbb{R}^{644} \to \mathbb{R}^{128} \to \mathbb{R}^{32} \tag{3}$$

Architecture: `Linear(644, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 32)`.

(Full-bank design: 644D. Santander 17-group implementation: Causal expert receives 129D via FeatureRouter; the compressor input dim is 129 in practice.)

Performing causal analysis directly on all raw features at full-bank scale would yield $644^2 \approx 414\{,\}000$ pairwise relationships; at 129D it is $129^2 \approx 16\{,\}600$. The Compressor summarizes these into 32 core causal variables, alleviating the computational burden of DAG learning.

Causal Intervention Operation

$$\hat{\boldsymbol{z}} = \boldsymbol{z} + \boldsymbol{z}(\boldsymbol{W} \circ \boldsymbol{W}) \tag{4}$$

Where:

- $\boldsymbol{W} \in \mathbb{R}^{32\times32}$: learnable weighted adjacency matrix (`nn.Parameter`)
- $\boldsymbol{W} \circ \boldsymbol{W}$: Hadamard square guaranteeing non-negative causal strength
- $W_{i,j}^2$: causal influence strength in the direction variable $j \to$ variable i
- residual connection ($\boldsymbol{z}+$): adds causally adjusted correction while preserving original information

This generates a corrected representation $\hat{\boldsymbol{z}}$ of each customer's latent characteristic vector $\boldsymbol{z}$, adjusted by the causal influence that other characteristics exert on it. The result is a causally adjusted representation, as opposed to a merely correlated one.

The adjacency matrix $\boldsymbol{W}$ is initialized at small scale via `torch.randn(32, 32) * 0.01` to ensure stability in the early phase of training.

Causal Encoder

Transforms the causally intervened representation into the final output dimension.

$$\boldsymbol{o} = \text{CausalEncoder}(\hat{\boldsymbol{z}}): \quad \mathbb{R}^{32} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{5}$$

Architecture: `Linear(32, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 64)` → `LayerNorm(64)` → `SiLU`.

The 64D output shares the same dimension as other experts (PersLay, DeepFM, etc.), enabling direct comparison and weighted summation in the CGC gate attention mechanism.

DAG Regularization Loss

Two regularization terms are used during training to maintain the DAG structure.

$$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \|\boldsymbol{W} \circ \boldsymbol{W}\|_1 \tag{6}$$

Hyperparameter	Default	Role
`dag_lambda`	`0.01`	Acyclicity constraint strength
`sparsity_lambda`	`0.001`	Adjacency matrix sparsity (L1)
`n_causal_vars`	`32`	Number of causal variables = number of DAG nodes

The acyclicity term $h(\boldsymbol{W})$ enforces the no-cycle constraint, while the sparsity term $\|\boldsymbol{W} \circ \boldsymbol{W}\|_1 = \sum_{i,j} W_{i,j}^2$ encourages retaining only a small number of meaningful causal relationships among the possible $32 \times 32 = 1\{,\}024$ edges.

Difference from the original paper. The original paper strictly satisfies the equality constraint $h(\boldsymbol{W}) = 0$ via an augmented Lagrangian, whereas this implementation relaxes it to a simple penalty $\lambda \cdot h(\boldsymbol{W})$. This is a practical choice to maintain balance with the other loss terms in MTL while achieving approximate acyclicity. Convergence to $h(\boldsymbol{W}) < 0.1$ at the end of training with $\lambda = 0.01$ is sufficient.

> **Caution:** Setting `dag_lambda` > 0.1 causes the adjacency matrix $\boldsymbol{W}$ to converge to the zero matrix, destroying the causal structure entirely. When the DAG regularization loss dominates the task loss, the expert degenerates into an identity function ($\hat{\boldsymbol{z}} \approx \boldsymbol{z}$).

Financial Domain Application

Learning Causal Directions Between Behaviors

In the financial domain, the Causal Expert learns causal directions among customer behavioral variables.

- **Confounder control**: From the correlation between "card usage and loyalty," the pure causal effect of card usage on loyalty is extracted by controlling for income as a confounding variable.
- **Intervention effect prediction**: Causal grounds for "how will customer behavior change if this benefit is provided" are internalized.
- **Interpretability**: The adjacency matrix $\boldsymbol{W} \circ \boldsymbol{W}$ can be extracted via `get_causal_graph()` and visualized as a heatmap to show which latent causal variables strongly influence others.

Causal Graph Extraction

```python
def get_causal_graph(self) -> torch.Tensor:
    """graph[i,j] = W[i,j]^2: causal strength of variable j -> variable i"""
    return (self.W * self.W).detach()
```

The returned `[32, 32]` matrix is used for causal path tracing, sparsity monitoring, and recommendation reason generation (LLM grounding).

Original Paper vs. Implementation Comparison

Item	Paper (Zheng et al.)	This Implementation
Purpose	Learn DAG structure from observational data	Learn causal relationships among features within an expert
Number of variables	Tens to hundreds (general-purpose)	Fixed at 32
Acyclicity	Augmented Lagrangian (equality constraint)	Simple penalty: $\lambda \cdot h(\boldsymbol{W})$
Adjacency matrix	$\boldsymbol{W}$ (sign-agnostic)	$\boldsymbol{W} \circ \boldsymbol{W}$ (non-negative)
Training	Independent optimization	End-to-end joint MTL training
Output	DAG adjacency matrix	64D causal representation + DAG (for visualization)

Optimal Transport Expert

Wasserstein Distance vs. KL Divergence

Geometric Perspective on Distribution Comparison

Traditional methods for measuring the difference between two probability distributions include:

- **KL divergence**: $D_{\mathrm{KL}}(P \parallel Q) = \sum_i P_i \log\left(\frac{P_i}{Q_i}\right)$
- **JS divergence**: a symmetric version of KL divergence
- **Total variation distance**: $\mathrm{TV}(P, Q) = \frac{1}{2}\sum_i |P_i - Q_i|$

These share a common limitation: they ignore the geometric structure of the underlying space. When distribution P is concentrated in "Seoul," Q in "Incheon," and R in "Busan," KL divergence and TV distance yield $\mathrm{dist}(P, Q) \approx \mathrm{dist}(P, R)$ — when the supports do not overlap, the distances are identical. Yet intuitively, Seoul–Incheon is closer than Seoul–Busan.

Advantages of Wasserstein Distance

The Wasserstein distance (earth mover's distance) reflects the distance structure of the underlying space.

$$W(\boldsymbol{\mu}, \boldsymbol{\nu}) = \min_{\boldsymbol{P} \in \mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C} \rangle_F \tag{7}$$

Where:

- $\boldsymbol{\mu}, \boldsymbol{\nu} \in \Delta^d$: source and target distributions (vectors on the probability simplex)
- $\boldsymbol{C} \in \mathbb{R}^{d \times d}$: cost matrix — $C_{i,j}$ is the cost of transporting one unit of mass from location i to j
- $\boldsymbol{P}$: transport plan — $P_{i,j}$ is the actual mass transported from i to j
- $\mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu}) = \{\boldsymbol{P} \geq 0 : \boldsymbol{P}\mathbf{1} = \boldsymbol{\mu}, \boldsymbol{P}^\top \mathbf{1} = \boldsymbol{\nu}\}$: marginal distribution constraints

Key reasons why Wasserstein distance is superior to KL divergence:

Property	KL Divergence	Wasserstein Distance
Non-overlapping distributions	Undefined when $Q_i = 0$ (∞)	Always finite
Geometric awareness	Ignores distances in the underlying space	Reflected via the cost matrix
Continuous deformation	May change discontinuously	Changes continuously with distribution shift
Transport plan	Provides only a scalar value	Provides how mass is transformed ($\boldsymbol{P}$)
Symmetry	Asymmetric ($D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$)	Symmetric (metric)

Sinkhorn Entropy Regularization

Computational Limitation of the Kantorovich Problem

The original Kantorovich optimal transport problem is a linear program (LP) with d^2 variables and $2d$ equality constraints, incurring a computational cost of $O(d^3 \log d)$. This is impractical for real-time inference.

Entropy Regularization

The key insight of Cuturi (NeurIPS 2013): adding an entropy term fundamentally changes the structure of the problem.

$$\min_{\boldsymbol{P} \in \mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C} \rangle + \varepsilon \cdot H(\boldsymbol{P}) \tag{8}$$

Where:

- $H(\boldsymbol{P}) = -\sum_{i,j} P_{i,j} \log P_{i,j}$: entropy of the transport plan
- $\varepsilon = 0.1$: regularization coefficient

Adding the entropy term makes the problem strictly convex with a unique solution. The optimal solution takes the form $\boldsymbol{P}^* = \operatorname{diag}(\boldsymbol{a}) \boldsymbol{K} \operatorname{diag}(\boldsymbol{b})$, where $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$ is the Gibbs kernel and $\boldsymbol{a}, \boldsymbol{b}$ are scaling vectors obtained via Sinkhorn alternating normalization.

Computational complexity reduces to $O\left(\frac{d^2}{\varepsilon^2}\right)$, making it amenable to GPU parallelization.

Intuition of Sinkhorn Algorithm

The Sinkhorn algorithm is an alternating projection:

1. Take the Gibbs kernel $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$ as the initial transport plan
2. Row normalization: adjust each row sum to match the source distribution $\boldsymbol{\mu}$
3. Column normalization: adjust each column sum to match the target distribution $\boldsymbol{\nu}$
4. Alternating row/column normalization converges to the optimal transport plan satisfying both marginal constraints simultaneously

Log-Domain Sinkhorn Algorithm

Numerical Stability Issues

Standard Sinkhorn operates directly on the elements of the Gibbs kernel $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$; when ε is small, $\exp\left(-\frac{C_{i,j}}{\varepsilon}\right)$ becomes extremely small, causing floating-point underflow.

Log-Domain Dual Variable Update

$$\boldsymbol{u}_{\text{new}} = \log \boldsymbol{\mu} - \operatorname{logsumexp}_j\left(-\frac{C_{i,j}}{\varepsilon} + v_j\right) \tag{9}$$

$$\boldsymbol{v}_{\text{new}} = \log \boldsymbol{\nu} - \operatorname{logsumexp}_i\left(-\frac{C_{i,j}}{\varepsilon} + u_i\right) \tag{10}$$

Where:

- $\boldsymbol{u}, \boldsymbol{v}$: log-domain dual variables
- $\log \boldsymbol{K} = -\frac{\boldsymbol{C}}{\varepsilon}$: log-domain Gibbs kernel
- `logsumexp`: $\log \sum_j \exp(a_j)$ — prevents overflow and underflow

Implementation note: In the theoretical definition, the $\boldsymbol{v}$ update uses $\boldsymbol{C}^\top$, but since the PSD cost matrix in this implementation ($\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M}$) is symmetric, $\boldsymbol{C}^\top = \boldsymbol{C}$. In code, the same result is obtained by changing the axis (dim) of `logsumexp` instead of transposing.

- Number of iterations: 10 (default, `sinkhorn_iterations`)

After convergence, the transport plan and Wasserstein distance are computed as:

$$\log \boldsymbol{P} = \boldsymbol{u} \oplus \log \boldsymbol{K} \oplus \boldsymbol{v}, \quad \text{i.e.,} \quad \log P_{i,j} = u_i + \log K_{i,j} + v_j \tag{11}$$

$$W(\boldsymbol{\mu}, \boldsymbol{\nu}_k) = \langle \boldsymbol{P}, \boldsymbol{C} \rangle_F = \sum_{i,j} P_{i,j} \cdot C_{i,j} \tag{12}$$

Learnable Reference Distributions and PSD Cost Matrix

Distribution Projection

Transforms customer features into a probability simplex.

$$\boldsymbol{\mu} = \text{softmax}(\text{DistProjector}(\boldsymbol{x})) \in \Delta^{32} \tag{13}$$

Architecture: `Linear(644, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 32)` → `softmax`. *(Full-bank design: 644D input. Santander 17-group: OT expert receives 95D via FeatureRouter.)*

Applying softmax ensures the 32 dimensions sum to 1, allowing each dimension to be interpreted as "the proportion of spending propensity allocated to a category."

Learnable Reference Distributions

$$\boldsymbol{\nu}_k = \text{softmax}(\ell_k) \in \Delta^{32}, \quad k = 1, ..., 16 \tag{14}$$

- ℓ_k: `nn.Parameter [16, 32]` — learnable logits
- Initialization: `torch.randn(16, 32) * 0.1`
- Each $\boldsymbol{\nu}_k$ learns the distribution of a typical customer type

The 16 reference distributions are not predefined; rather, they capture naturally emerging cluster types as the data is trained. For example, $\boldsymbol{\nu}_3$ may learn to represent a "travel-centric spender" type, while $\boldsymbol{\nu}_7$ may represent a "cost-conscious household" type.

PSD Cost Matrix

$$\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32 \times 32} \tag{15}$$

- $\boldsymbol{M}$: `nn.Parameter [32, 32]` — learnable basis matrix
- Initialization: $\boldsymbol{I} + \mathcal{N}(0, 0.01)$

PSD guarantee: $\boldsymbol{x}^\top (\boldsymbol{M}^\top \boldsymbol{M}) \boldsymbol{x} = \|\boldsymbol{M}\boldsymbol{x}\|^2 \geq 0$. Non-negativity of costs is guaranteed automatically, eliminating the physically nonsensical scenario of "gaining value by transporting." Since the cost is learnable, the model learns task-specific semantic distances — e.g., "food → dining out is low-cost, food → travel is high-cost."

Wasserstein Distance Vector

$$\boldsymbol{w} = [W(\boldsymbol{\mu}, \boldsymbol{\nu}_1), W(\boldsymbol{\mu}, \boldsymbol{\nu}_2), ..., W(\boldsymbol{\mu}, \boldsymbol{\nu}_{16})] \in \mathbb{R}^{16} \tag{16}$$

Each customer's spending distribution $\boldsymbol{\mu}$ is compared against 16 representative customer types, yielding a 16-dimensional distance vector. This constitutes a distributional coordinate system — the customer is positioned by their distances from 16 reference points.

Wasserstein Encoder

$$\boldsymbol{o} = \text{WassersteinEncoder}(\boldsymbol{w}): \quad \mathbb{R}^{16} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{17}$$

Architecture: `Linear(16, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 64)` → `LayerNorm(64)` → `SiLU`.

Financial Domain Application

Distributional Shift Between Segments

In the financial domain, the OT Expert performs structural matching between a customer's spending pattern distribution and typical profile distributions.

The Wasserstein distance quantifies "how different this customer's spending pattern is from a typical travel / savings / dining-out profile, and in which direction which categories must shift for the patterns to align." The transport plan $\boldsymbol{P}$ provides not merely a scalar distance but a concrete transformation direction.

Original Paper vs. Implementation Comparison

Item	Paper (Cuturi 2013)	This Implementation
Distribution input	Fixed discrete/continuous distributions	Learned softmax distributions (32D)
Cost matrix	Fixed (Euclidean, etc.)	Learnable PSD: $\boldsymbol{M}^\top \boldsymbol{M}$
Reference distribution	Single target distribution	16 learnable prototype distributions
Sinkhorn implementation	Standard domain (matrix multiplication)	Log-domain (numerical stability)
Number of iterations	Until convergence	Fixed at 10
Output	Scalar OT distance	16D distance vector → 64D encoding

FP16 Numerical Stability

Challenges of Mixed Precision (AMP) Environment

Enabling AMP (Automatic Mixed Precision) on the g4dn T4 GPU approximately doubles training speed, but the limited representable range of FP16 ($\approx 6 \times 10^{-8}$ to 6.5×10^4) introduces numerical stability issues.

Causal Expert FP16 Issues

Overflow in Taylor Approximation

In the NOTEARS Taylor 10-term approximation, the accumulated matrix powers $\boldsymbol{M}^k$ can grow rapidly in magnitude as k increases. When values exceed 6.5×10^4 in FP16, they become `inf`, which propagates as NaN through the `trace` operation.

Solution:

- Keep the initialization scale of $\boldsymbol{W}$ at `0.01` so that elements of $\boldsymbol{W} \circ \boldsymbol{W}$ remain on the order of 10^{-4}
- Prevent rapid growth of $\boldsymbol{W}$ elements via gradient clipping (`gradient_clip_norm: 5.0`)
- It is recommended to cast the DAG regularization computation to `torch.float32`

Adjacency Matrix Element Explosion

When `dag_lambda` is too small, the acyclicity constraint weakens, allowing $\boldsymbol{W}$ elements to grow large, which triggers overflow in the higher-order terms of the Taylor approximation.

OT Expert FP16 Issues

Issues in Sinkhorn Log-Domain

The `logsumexp` operation in log-domain Sinkhorn can cause the following issues in FP16:

1. **Softmax probability underflow**: Very small probability values from `F.softmax(dist_logits, dim=-1)` fall below the FP16 minimum and become 0. Subsequently, `1e-8` in `torch.log(a.clamp(min=1e-8))` is not exactly representable in FP16.
2. **Cost matrix scale**: In computing $-\frac{\boldsymbol{C}}{\varepsilon}$ with $\varepsilon = 0.1$, cost values are amplified by a factor of 10. Exceeding the FP16 range causes NaN.

Solution:

- Change `clamp(min=1e-7)` to `clamp(min=1e-6)`. Since Sinkhorn internals are cast to FP32, `1e-6` is safely representable in FP32.
- Perform Sinkhorn internal operations in `torch.float32` and cast only the output back to FP16
- Use the `@torch.cuda.amp.custom_fwd(cast_inputs=torch.float32)` decorator

Phase 2 NaN Fix Experience

NaN issues encountered during actual training and their resolutions:

Expert	Symptom	Cause and Resolution
Causal	`causal_loss` becomes NaN after epoch 3	$\boldsymbol{W}$ elements exceed FP16 range. Taylor higher-order term overflow. → Cast DAG regularization to FP32, reduce grad clip from 5.0 to 1.0
OT	NaN propagation after Sinkhorn iteration 5	Cost scale amplification in $-\frac{\boldsymbol{C}}{\varepsilon}$. → Perform Sinkhorn internals in FP32, verify `cost_matrix` initialization $\boldsymbol{I} + \mathcal{N}(0, 0.01)$
OT	Intermittent NaN in specific batches only	Extreme one-hot distribution in softmax output, `log(0)` propagation. → Apply `clamp(min=1e-6)` (safe in FP32 after casting)

FP16 Principle: For both experts, all core numerical operations (Taylor matrix exponential, Sinkhorn log-domain) are performed in FP32; only the input and output tensors are cast to FP16. AMP's `GradScaler` automatically skips steps where NaN gradients are detected, but frequent skips degrade training quality — the root cause must be addressed.

Implementation Notes

Common Design Patterns of Both Experts

Item	Causal Expert	OT Expert
Registry Name	`"causal"`	`"optimal_transport"`
Input Dimension	129D (demographics 11D + product_holdings 24D + txn_behavior 14D + derived_temporal 4D + product_hierarchy 34D + gmm 22D + rolling_stats 20D)	95D (demographics 11D + product_holdings 24D + txn_behavior 14D + derived_temporal 4D + gmm 22D + rolling_stats 20D)
Latent Space	32D (causal variables)	32D (probability simplex)
Output Dimension	64D	64D
Core Mechanism	SCM: $\hat{\boldsymbol{z}} = \boldsymbol{z} + \boldsymbol{z}(\boldsymbol{W} \circ \boldsymbol{W})$	Sinkhorn: $W(\boldsymbol{\mu}, \boldsymbol{\nu}_k) \times 16$
Learnable Parameters	$\boldsymbol{W}$ `[32,32]` adjacency matrix	`reference_logits [16,32]` + `cost_matrix [32,32]`
Regularization Loss	$\mathcal{L}_{\text{DAG}}$ (acyclicity + sparsity)	None (entropy regularization is implicit)
Interpretable Output	4D (causal strength, etc.)	4D (transport cost, etc.)

PLE Integration

Both experts are integrated into PLE via the same pathway:

```
# ple_cluster_adatt.py
elif name in ("causal", "optimal_transport"):
    out, _ = expert(inputs.features[:, expert_slice])  # FeatureRouter slices per-expert subset
```

Only the Causal Expert carries an additional regularization loss:

```
# ple_cluster_adatt.py -- compute_loss
if self.training and "causal" in self.shared_experts:
    dag_reg = self.shared_experts["causal"].get_dag_regularization()
    total_loss = total_loss + dag_reg
```

The OT Expert has no separate regularization loss; `reference_logits` and `cost_matrix` are naturally trained via backpropagation through the task loss.

Mathematical Perspective Comparison of Three Experts

Although DeepFM, Causal, and OT all receive overlapping but distinct feature subsets as input (Causal: 129D, OT: 95D in the 17-group Santander implementation, via FeatureRouter), the mathematical structures they extract are fundamentally different:

Expert	Extraction Target	Mathematical Property	Unique Contribution
DeepFM	Symmetric pairwise feature interactions $\langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle$	Commutative ($i \leftrightarrow j$)	Captures second-order crossings in $O(nk)$
Causal	Directional causality $W_{i,j}^2$	Asymmetric, acyclic (DAG)	Confounder removal, causal direction
OT	Distributional distance $W(\boldsymbol{\mu}, \boldsymbol{\nu}_k)$	Distance function (metric)	Geometric position of distributions

Gradient Flow

Causal Expert

```
total_loss
  |-- task_loss -> causal_encoder -> _apply_causal_mechanism
  |                                     -> W (nn.Parameter) <- gradient
  |-- dag_loss -> get_dag_regularization()
                    -> W (nn.Parameter) <- gradient
```

$\boldsymbol{W}$ receives gradients from both the task loss and the DAG regularization loss. The balance between the two gradients is controlled by the `dag_lambda` setting.

OT Expert

```
total_loss -> wasserstein_encoder -> _sinkhorn_distance -> dist_projector
                                        |-- cost_matrix <- gradient
                                        |-- reference_logits <- gradient
```

Sinkhorn's 10 iterations use unrolled gradients. A larger number of iterations lengthens the gradient chain, increasing the risk of vanishing/exploding gradients; therefore, gradient clipping (`gradient_clip_norm: 5.0`) should be used in conjunction.

Debugging Guide

Causal Expert Issues

Symptom	Cause	Action
DAG collapse ($\boldsymbol{W} \approx 0$)	`dag_lambda` too large (> 0.1)	Reduce `dag_lambda` to 0.01 or below
Acyclicity violation ($h(\boldsymbol{W}) \gg 0$)	`dag_lambda` too small or learning rate too large	Gradually increase `dag_lambda`, reduce learning rate

Symptom	Cause	Action
`causal_loss` NaN	Taylor approximation divergence, $\boldsymbol{W}$ elements too large	Strengthen gradient clip ($5.0 \to 1.0$), cast to FP32
Expert output is constant	$\boldsymbol{W}$ training stalled	Set per-expert learning rate, verify initialization

OT Expert Issues

Symptom	Cause	Action
Sinkhorn divergence (NaN/Inf)	ε too small (< 0.01) or cost scale too large	Set $\varepsilon \geq 0.1$, verify cost initialization
Degenerate distribution (one-hot softmax)	`dist_projector` logits too large	Apply temperature scaling or increase dropout
All distances identical	Reference distribution collapse or zero cost matrix	Monitor diversity of `reference_logits`
Negative OT distance	PSD guarantee on cost matrix violated	Verify `cost_matrix.T @ cost_matrix`

Further Reading

- **NOTEARS / DAGMA**: Zheng et al. NeurIPS 2018; Bello et al. ICML 2022.
- **Sinkhorn OT**: Cuturi NeurIPS 2013; Sinkhorn 1964; Villani 2009 (*Optimal Transport: Old and New*).
- **Causal inference foundations**: Pearl 2009 (*Causality*, 2nd ed.); Rubin 1974.